

Week 6 - Association Rules Mining

 Copy 

 **The deadline for the Week 5 lab assessment is Thursday, 2nd April. You are still welcome to present your Week 5 lab results to *ANY* lab facilitator at *ANY* lab session in Week 6 before the deadline.**

Lab Objectives

- Find frequently occurring itemsets using the Apriori algorithm.
- Compute the support of the frequent itemset.
- Compute the confidence and lift of an association rule.

Technologies Covered

- Python `mlxtend`
- Python `pandas`
- Python `numpy`
- Jupyter Notebook

Algothrims for Association Rules Mining

FP-Growth and ECLAT algorithms are not assessed in the final exam.

Apriori algorithm

FP-Growth algorithm

ECLAT algorithm

The Apriori algorithm works by first identifying the frequent itemsets in a dataset. Any subset of a frequent itemset must also be frequent.

Terminate when no frequent or candidate set can be generated. The Apriori algorithm uses a bottom-up approach, starting with individual items and gradually building up to more complex itemsets (Ali, 2023).

Comparison of These Three Algorithms

This part will not be accessed in the final exam.

Algorithm	Memory Usage	Runtime Efficiency*	Implementation Complexity
Apriori	Intensive	Can be slow, especially for large datasets.	Easy to implementation
FP-Growth	Less memory compared to Apriori	Generally faster than Apriori.	More complex to implement.
ECLAT	Memory usage is significantly lower than Apriori.	Runtime efficiency is usually between Apriori.	Implementation complexity is moderate.

*: Some research studies suggest that ECLAT is generally faster than FP-Growth, and the performance can vary depending on the characteristics of the datasets being analysed.

In this lab, we will use the *Apriori algorithm* to mine rules.

Key Measures for Association Rule Mining

Support

Confidence

Lift

The probability that a transaction contains $X \cup Y$.

$$\text{Support}(X \Rightarrow Y) = P(X \cup Y)$$

$$\text{Support}(\{X\} \rightarrow \{Y\}) = \frac{\text{Transactions containing both } X \text{ and } Y}{\text{Total number of transactions}}$$

Association Rules Mining with Python

There are various software tools that can be used to perform association rules mining, such as Python, R, Weka, or Microsoft SQL Server Analysis Services (MS SSAS). In this lab, we use Jupyter Notebook and Python libraries to mine rules.

Download of the Dataset

Download the dataset for this lab. Note: download the dataset to your working directory. In this dataset, the column, "pep" indicates whether the customer purchased a Personal Equity Plan after the most recent promotional campaign.



54KB

bank-data.csv

Download

Open

The dataset for this lab

Code Execution

For this lab, you have two options to execute the code: a local solution or a Docker solution. You can choose **either** option.

- **Local solution**

- Run Jupyter Notebook locally

```
#Check your current working directory
import os
os.getcwd()

#If you need to change your current working directory, you can
os.chdir('working directory path')
```

- **Docker Solution**

- If you run this lab inside Docker, please **save the dataset in the "notebooks" directory within our GitHub repository.**

Data Pre-process

- If you haven't installed `mlxtend`, you need to install `mlxtend` first.

```
#Install mlxtend library.
pip install mlxtend
```

- Import necessary libraries

```
#Import necessary libraries, pandas, numpy and mlxtend
import pandas as pd
import numpy as np
import mlxtend
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
```

- Import the dataset to Jupyter Notebook

```
#Read the csv file
df = pd.read_csv('bank-data.csv')

#print the df
print(df)
```

- Descriptive statistics

```
#Show the data head
df.head()

#Check the data types in the dataframe
df.dtypes

#The columns of the dataframe
df.columns

#Check the number of rows and columns
df.shape
```

- Check missing values in the dataframe

```
#Check missing values in the dataframe
df.isna()

#Count total missing values at each column in the dataframe
df.isna().sum()
```

We can see, that there are no missing values in the data frame.

- The column, "id", is not useful for association rules mining, therefore, the column needs to be dropped.

```
#Drop the column, 'id'.
new_df = df.drop('id',axis = 'columns')
```

- Data transformation

```
#Data transformation
#TransactionEncoder() function only can handle string type
new_df = new_df.astype(str)

#TransactionEncoder() was designed to covert lists to array
list = new_df.values.tolist()

#Covert the list to one-hot encoded boolean numpy array.
#Apriori function allows boolean data type only, such as 1 and 0,
or FALSE and TRUE.
te = TransactionEncoder()
array_te = te.fit(list).transform(list)

#Check the array
array_te

#Check the columns
te.columns_

#Apriori function can handle dataframe only, covert the array to
a dataframe
arm_df = pd.DataFrame(array_te, columns = te.columns_)
```

- **Is the provided data transformation solution good? If not, could you propose a better solution for data transformation?**
- **How are numerical attributes handled in your Project 1 for association rule mining? Could you justify your solution?**

Association Rules Mining with `m1xtend` Library

Note: All thresholds for confidence, lift, and support were not discussed in this part. This section serves as an example to help you become familiar with mining association rules using the `m1xtend` library. The thresholds mentioned here are not intended as references for your Project 1. In Project 1, you may need to discuss the selection of appropriate thresholds.

We don't explain rules/results/outputs in the lab sheet, as explanating rules is a task in Project 1. Please research independently to understand how to explain rules in plain English.

- `min_support`: a minimum support threshold, used to filter out itemset that don't occur frequently enough.

```
#Find the frequent itemsets
frequent_itemsets = apriori(arm_df,min_support=0.2,use_colnames=True)

#Check the length of rules
frequent_itemsets['length']=frequent_itemsets['itemsets'].apply(lambda x: len(x))

#Assume the length is 2 and the min support is >= 0.3
frequent_itemsets[(frequent_itemsets['length']==2) &
                  (frequent_itemsets['support']>=0.3)]
```

- Use confidence to filter out association rules that are not strong enough.

```
#Assume the min confidence is 0.5
rules_con = association_rules(frequent_itemsets,
metric="confidence",min_threshold=0.5)
```

- Use lift to filter out association rules

```
#Assume the min lift is 1
rules_lift = association_rules(frequent_itemsets,
metric="lift",min_threshold=1)

#Based on min confidence (=0.5),
#output antecedents, consequents, support, confidence and lift.
result_arm =
rules_con[['antecedents','consequents','support','confidence','lift']]
```

- Filter the rules whose confidence ≥ 0.7

```
#Find the rules whose confidence >= 0.7
new_result_arm = result_arm[result_arm['confidence']>=0.7]
```

[Optional] Association Rules Mining without Python Libraries

 You can also mine association rules without Python libraries, except `pandas` and `numpy`.

 Hints:

- Recall apriori algorithm
 1. Scan the database once to get frequent 1-itemset; $k=1$
 2. Repeat
 - a. Generate length $(k+1)$ candidate itemsets from length k frequent itemsets
 - b. Test the candidates against the database to find frequent $(k+1)$ itemsets
 - c. Set $k=k+1$
 3. Terminate when no frequent or candidate set can be generated
 4. Return all the frequent itemsets
- Pseudocode

```

procedure Apriori ( $T$ , minSupport) { //  $T$  is the database and min-Support
  is the minimum support
   $C_k$ : Candidate itemset of size  $k$ 
   $L_k$ : frequent itemset of size  $k$ 
   $L1 = \{\text{frequent items}\};$ 
  for ( $k = 1$ ;  $L_k \neq \emptyset$ ;  $k++$ ) do begin
     $C_{k+1}$  = candidates generated from  $L_k$ ;
    for each transaction  $t$  in database do
      increment the count of all candidates in  $C_{k+1}$  that are contained in  $t$ 
     $L_{k+1}$  = candidates in  $C_{k+1}$  with min_support
  end
  return  $\bigcup_k L_k$ ;

```

From: *Pesudo code for Apriori algorithm*, by Jumaah, Al-Janabi, and Ali, 2014, Flickr
<https://www.iasj.net/iasj/download/ea6e857c5fed9a37> ↗

- You may need to use `for` and `while` loops in your code.

- You need to write your code to compute confidence, support and lift for your rules.

References

Ali, M. (2023, January). *Association Rule Mining in Python Tutorial*. datacamp. <https://www.datacamp.com/tutorial/association-rule-mining-python> ↗

Li, H., Wang, Y., Zhang, D., Zhang, M., & Chang, E. Y. (2008, October). Pfp: parallel fp-growth for query recommendation. In *Proceedings of the 2008 ACM conference on Recommender systems*, 107-114.

Jumaah, A. K., Al-Janabi, S., & Ali, N. A. (2014). Hiding Sensitive Association Rules Over Privacy-Preserving Distributed Data Mining. *Kirkuk University Journal-Scientific Studies*, 9(1), 59-72.

Wang, L., Guo, Y., Guo, Y., Xia, X., Zhang, Z., & Cao, J. (2023). An Improved Eclat Algorithm based Association Rules Mining Method for Failure Status Information and Remanufacturing Machining Schemes of Retired Products. *Procedia CIRP*, 118, 572-577.

[Previous](#)

[Week 5 - Mini End-to-End Data Warehouse Project](#)

Last updated 11 days ago